Mixture of Experts and Mixture Density Networks

Ying JIN and Gloire LINVANI — MVA Master's

December 17, 2025

1 Introduction

Standard neural network regression minimizes mean squared error (MSE), which yields predictions approximating the conditional expectation $\mathbb{E}[y \mid x]$ [1]. While appropriate for unimodal output distributions, this approach fails for **one-to-many mappings** where a single input can correspond to multiple valid outputs. In such cases, the conditional mean may lie in low-density regions between modes, producing unrealistic predictions. This limitation is critical in inverse problems such as robot kinematics, image colorization, motion generation, etc.

Mixture of Experts (MoE) models [2, 3] address this challenge by combining multiple subnetworks ("experts") with a gating function that, for each input x, performs conditional computation by selecting (or weighting) the experts to use. Mixture Density Networks (MDNs) can be viewed as a probabilistic instantiation of this idea: a neural network parameterizes an input-dependent Gaussian mixture by outputting, for each component, the mixing coefficients, means, and variances. By leveraging the universal approximation properties of finite Gaussian mixtures [5], MDNs can represent a wide class of conditional distributions.

This work is primarily inspired by *Mixture Density Networks* by Christopher M. Bishop [1]. We first replicate the inverse-function toy example from the paper to illustrate the advantage of MDNs in one-to-many mappings, in contrast to a standard multilayer perceptron. We then study how the number of mixture components K affects expressiveness and robustness when the training data are contaminated with outliers. Motivated by the MoE paradigm, we design an MDN variant aimed at reducing inference cost from an engineering perspective. As a fifth experiment, we explored extending the MDN/MoE framework to text-conditioned human motion generation with diffusion models, formulating an MDN-based denoising head to better capture the multi-modality of $p(x_0 \mid c)$; due to compute and time constraints, this experiment is presented as a technically grounded proposal and partial implementation plan rather than a fully evaluated empirical study.

2 Mixture of Experts and Mixture Density Networks

2.1 Mixture of Experts Framework

Our goal is to model complex output distributions conditioned on an input. The MoE framework provides a principled approach to this problem by decomposing prediction into simpler subproblems handled by specialized experts. We introduce a discrete latent variable $z \in 1, \ldots, K$ indicating the active expert, which yields the conditional density:

$$p(y \mid x) = \sum_{k=1}^{K} p(y \mid x, z = k)\, p(z = k \mid x) \tag{1}$$

where $p(y \mid x, z = k)$ represents the k-th expert's conditional distribution, and $p(z = k \mid x)$ is the gating function that determines the input-dependent probability of selecting expert k. In

traditional MoE architectures, each expert is implemented as a separate neural network $f_k(x;\theta_k)$, and the gating function is learned via a separate gating network.In practice, MoE models can be trained either by standard stochastic gradient descent on the negative log-likelihood, or by viewing the expert index as a latent variable and applying an EM algorithm in suitable cases.

2.2 Mixture Density Networks

A natural way to instantiate this general MoE structure is to choose each expert to be a Gaussian density. Because finite Gaussian mixtures are universal density approximators: with enough components and suitably chosen parameters, they can approximate any continuous density arbitrarily well. Concretely, a MDN parameterizes the conditional density as:

$$p_\theta(y \mid x) = \sum_{k=1}^{K} \pi_k(x;\theta)\,\mathcal{N}\Big(y \mid \mu_k(x;\theta),\, \sigma_k^2(x;\theta)\, I\Big) \tag{2}$$

where $\pi_k(x;\theta) \equiv p_\theta(z = k \mid x)$ are the mixing coefficients((also referred to as the mixture weights), and $\{\mu_k(x;\theta), \sigma_k(x;\theta)\}$ are the mean and standard deviation of the k-th Gaussian component. A feed-forward neural network with parameters θ maps x to three types of parameters.
The network produces unconstrained outputs $a_k(x;\theta), , m_k(x;\theta), , s_k(x;\theta)k = 1^K$. To obtain valid mixture parameters, we apply the following transformations:

$$\pi_k(x;\theta) = \frac{\exp(a_k(x;\theta))}{\sum_{j=1}^{K} \exp(a_j(x;\theta))} \quad \text{(softmax)} \tag{3}$$

$$\mu_k(x;\theta) = m_k(x;\theta) \tag{4}$$

$$\sigma_k(x;\theta) = \exp(s_k(x;\theta)) \quad \text{(ensuring } \sigma_k > 0) \tag{5}$$

The softmax function in (3) guarantees that $\sum_{k=1}^{K} \pi_k(x;\theta) = 1$ and $\pi_k(x;\theta) \geq 0$, while the exponential transformation in (5) enforces positivity of the standard deviations.
Given a dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N}$, maximum-likelihood training amounts to minimising the negative log-likelihood(NLL), the loss function is expressed as:

$$\mathcal{L}_{\text{MDN}}(\theta) = -\sum_{n=1}^{N} \log p_\theta(y_n \mid x_n) = -\sum_{n=1}^{N} \log \sum_{k=1}^{K} \pi_k(x_n;\theta)\,\mathcal{N}\Big(y_n \mid \mu_k(x_n;\theta), \sigma_k^2(x_n;\theta) I\Big) \tag{6}$$

The gradient of $\mathcal{L}_{\text{MDN}}$ with respect to θ can be computed efficiently via backpropagation using the chain rule. Full derivations are provided in Appendix B.
In our experiments, the datasets exhibit one-to-many mappings. To highlight the MDN's ability to represent multiple outputs, we therefore generate predictions by sampling:

$$k \sim \text{Categorical}\big(\pi_1(x;\theta), \ldots, \pi_K(x;\theta)\big), \qquad y \sim \mathcal{N}\big(\mu_k(x;\theta),\, \sigma_k^2(x;\theta) I\big).$$

3 Experiments

Four experiments were conducted on synthetic datasets. The Model information is in Appendix A.

3.1 Experiment 1: MLP vs MDN in inverse Mapping (Bishop's Toy Problem)

Setup. Data: $x = y + 0.3\sin(2\pi y) + \epsilon$ with $y \sim \mathcal{U}(0,1)$, $\epsilon \sim \mathcal{U}(-0.1, 0.1)$ [1]. The forward mapping $y \mapsto x$ is single-valued; the inverse $x \mapsto y$ is multi-valued (up to 3 branches). We train MLP-Forward, MLP-Inverse, and MDN-Inverse ($K = 3$) on $N = 1000$ samples.

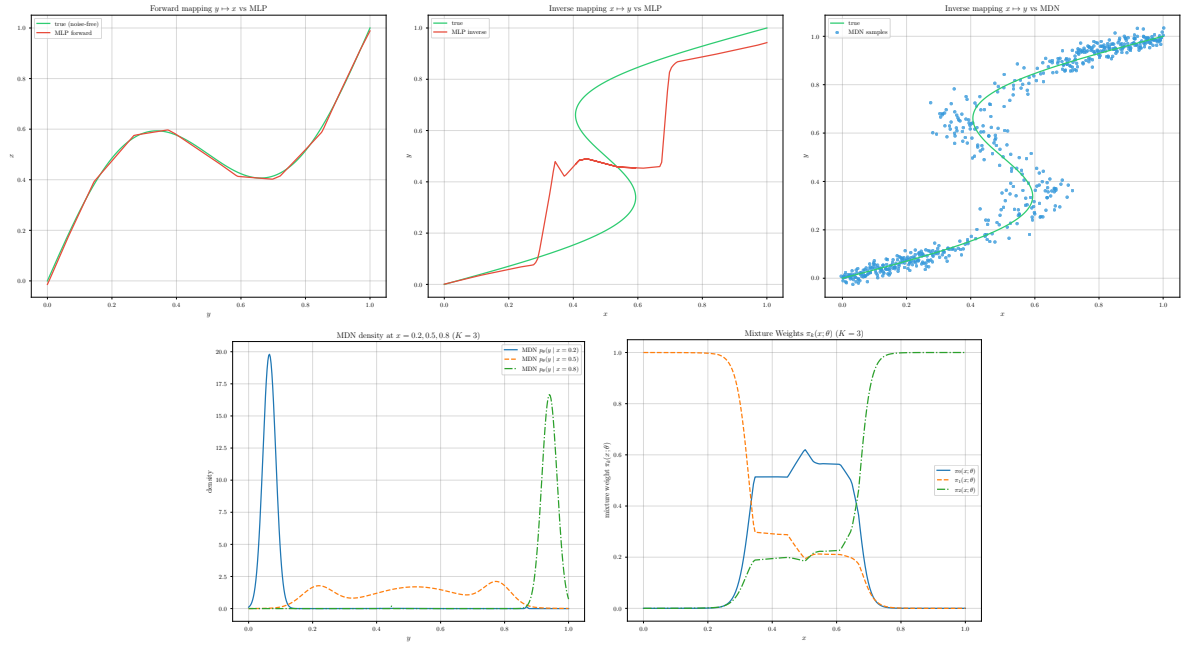


Figure 1: Experiment 1 (inverse-function toy problem). **(a)** MLP on the forward mapping. **(b)** MLP on the inverse mapping. **(c)** MDN samples. **(d)** Conditional densities $p_\theta(y \mid x)$ at $x \in \{0.2, 0.5, 0.8\}$. **(e)** Mixture weights $\pi_k(x)$

Results. Figure 1 shows that the MLP performs well on the single-valued forward mapping, but yields unrealistic averaged predictions on the inverse problem. In contrast, the MDN captures the underlying tri-modal structure. In Fig. 1(d), at $x = 0.5$ the model assigns significant probability mass to three distinct outputs, which is reflected by three clear peaks in the conditional density $p_\theta(y \mid x = 0.5)$. For $x \approx 0.2$ and $x \approx 0.8$, the inverse mapping has a single branch; accordingly, the conditional distribution is close to unimodal, with a small predicted variance. This behavior is also visible in the mixing weights in Fig. 1(e): around $x = 0.5$ several components are active, whereas as $x \to 0$ or $x \to 1$ the mixture effectively collapses to a single Gaussian component.

3.2 Experiment 2: Study the Number of Mixture Components K

Setup. We consider the same inverse toy problem but add heteroscedastic noise with input-dependent standard deviation

$$\sigma(y) = 0.1\big(0.1 + |y - 0.5|\big).$$

We train MDNs(2 hidden layers with 64 hidden units each)with $K \in \{1, 2, 3, 4, 5, 6, 10\}$. For each K, we select the best checkpoint via early stopping on a held-out validation negative log-likelihood (NLL), and report the corresponding test NLL.

Results. First, the MDN is able to capture heteroscedasticity: the predicted scales vary with the input and increase in regions where the noise level is higher. Second, for Bishop's toy example, $K = 3$ is sufficient to represent the three branches of the inverse mapping. Interestingly, even with $K = 2$ the generated samples already resemble the data qualitatively. When using $K = 4$ or $K = 5$, we observe that only about three components are effectively used, while the redundant components receive near-zero mixing weights and become inactive. Increasing K further (e.g., $K = 10$) does not noticeably improve the test NLL; however, we observe that more than three components become active. A possible explanation is that, with a large number

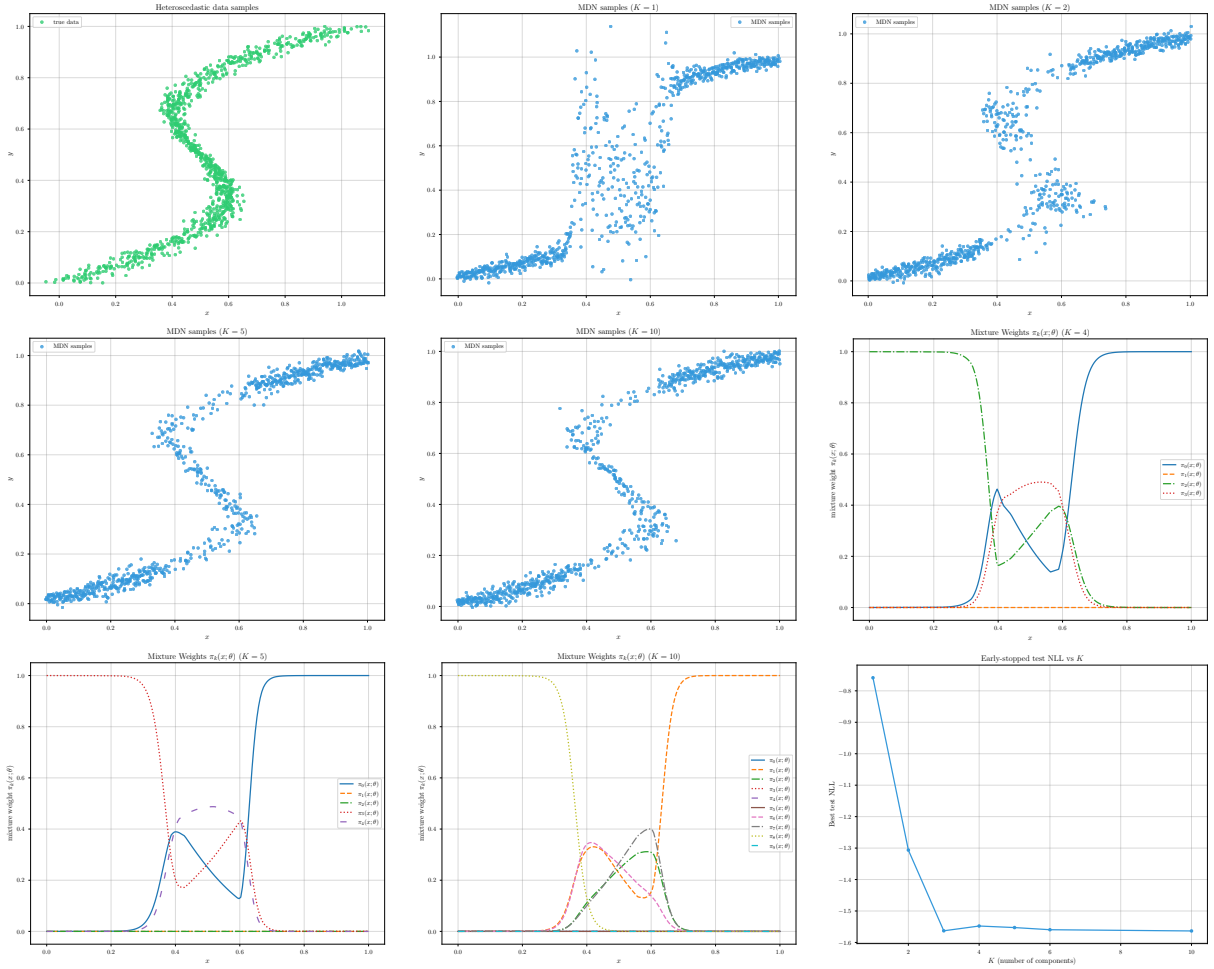


Figure 2: Experiment 2 (heteroscedastic inverse problem). From left to right: (a) ground-truth noisy data; MDN samples for (b) $K = 1$, (c) $K = 2$, (d) $K = 5$, and (e) $K = 10$; mixture weights for (f) $K = 4$, (g) $K = 5$, and (h) $K = 10$; and (i) test NLL versus K (performance saturates around $K = 3$).

of components, the model can *split* a branch into multiple Gaussians to better approximate deviations from a single Gaussian (e.g., heavy tails induced by heteroscedastic noise), or to allocate small components to low-density regions. This behavior is consistent with the non-identifiability of mixture models and the optimization dynamics under softmax competition. Finally, larger K increases the number of parameters and the inference cost; from an engineering standpoint, choosing K close to the effective number of modes can therefore reduce computation without sacrificing performance.

3.3 Experiment 3:Effect of K Under Outlier Contamination

Setup. Multi-branch data with 3 input-dependent branches (sinusoidal, linear) and heteroscedastic noise. We inject 0.5% outliers ($y \sim \mathcal{U}(-1, 1.5)$) into training and evaluate on clean test data. MDNs trained with $K \in \{3, 5, 7, 10\}$.

We observed that when $K = 10$, the model activates more mixture components. This raises the question of whether a large K may encourage the MDN to model rare outliers as additional modes. To test this hypothesis, we consider a variant of the dataset containing a small fraction of outliers (0.05%), highlighted in Fig. 4(a).

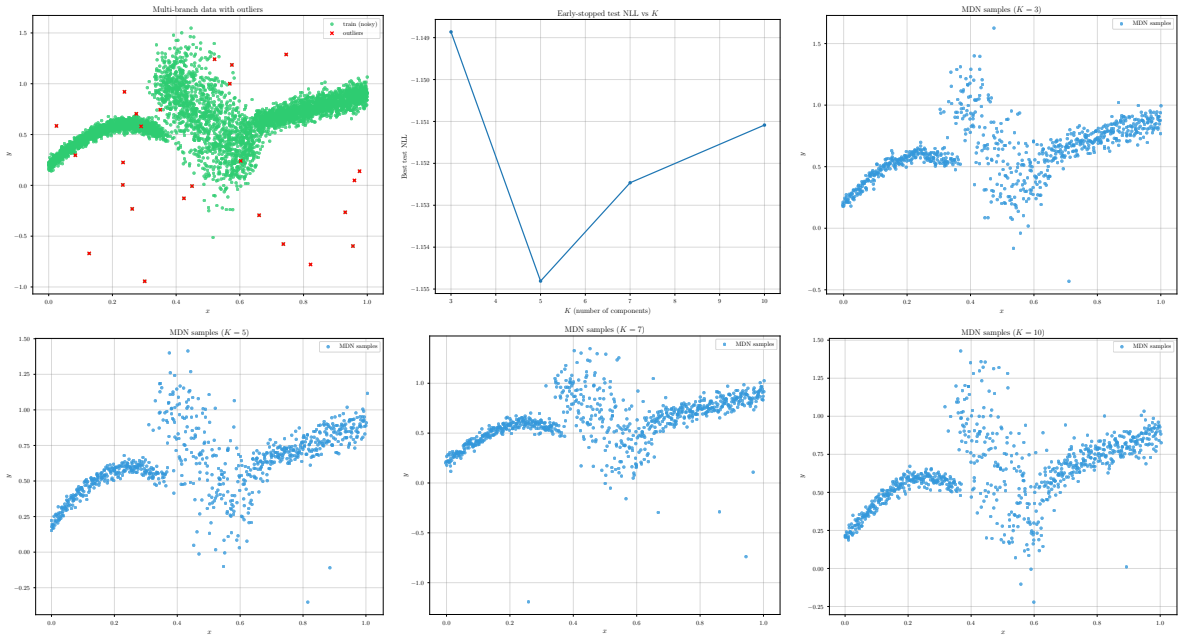


Figure 3: Experiment 3 (outlier contamination). From left to right, top to bottom: (a) dataset with 0.5% outliers (red crosses), (b) test NLL versus K, and MDN samples for (c) $K = 3$, (d) $K = 5$, (e) $K = 7$, and (f) $K = 10$.

Results. Figure 3 shows that the test NLL decreases up to $K = 5$ and then increases, suggesting overfitting for larger K. This trend is also visible in the sampling results: starting around $K = 7$, the model begins to assign non-negligible probability mass to the outlier region (bottom-left), effectively dedicating one or more components to fitting the outliers.

3.4 Experiment 4: GatedDual Architecture

Setup. 2D density modeling: star (hexagram, 1200 pts) + circle (800 pts) with uniform jitter. We compare MDN ($K = 8$, 22,792 params) vs. GatedDual ($K_A = 2$, $K_B = 6$, 22,506 params).

Results. A key advantage of MoE models is sparse activation: for a given input, only a subset of experts needs to be evaluated. In contrast, a standard MDN predicts the parameters of all K mixture components using a single shared network, which makes inference cost scale with K. Inspired by this MoE principle, we propose **GatedDual**, in which three networks are trained jointly: (i) a gating network that outputs the mixture weights $\{\pi_k(x;\theta)\}_{k=1}^{K}$, and (ii) two expert networks that predict means and scales for disjoint subsets of components.
Empirically, both the baseline MDN and GatedDual generate samples that match the data distribution well, but GatedDual produces visually cleaner samples and achieves a lower NLL. We use early stopping on a validation set with a patience of 200 epochs; the baseline MDN typically stops substantially earlier than GatedDual, suggesting that GatedDual continues improving for longer before overfitting.
At inference time, given an input x, we first evaluate the gating network to obtain $\{\pi_k(x;\theta)\}$. We then select a component index (e.g., by sampling $k \sim \text{Categorical}(\pi(x;\theta))$ or taking $k = \arg\max_k \pi_k(x;\theta)$) and evaluate only the corresponding expert to obtain $(\mu_k(x;\theta), \sigma$
Figure 4 shows GatedDual produces sharper samples. The structure provides inductive bias: simple modes (circle) are routed to Expert A, complex modes (star) to Expert B.

Table 1: Experiment 4: MDNFull vs. GatedDual comparison.

Model	Best Test NLL	FLOPs	Params
MDNFull	−7.61	22,464	22,792
GatedDual	**−11.14**	**20,304** (avg)	22,506

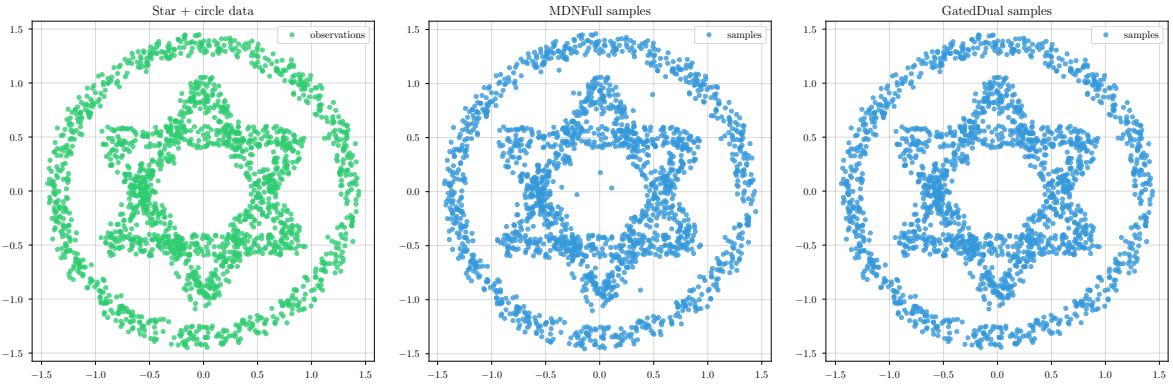


Figure 4: Experiment 4. **Left:** Star+circle data. **Center:** MDN samples. **Right:** GatedDual samples (sharper edges).

4 Discussion

Our experiments support the standard view that Mixture Density Networks are a natural fit for one-to-many regression. While a deterministic MLP is forced to collapse multiple targets into a single averaged prediction, the MDN represents a full conditional density and can generate diverse outputs. In our 1D and 2D toy settings, the MDN additionally adapts its predicted scales under heteroscedastic noise and we believe that it can applied into more complex examples.

The choice of the number of mixture components K is priority in practice. If K is too small, the model lacks capacity to represent all modes and sample quality degrades. If K is large, inference becomes more expensive and the model is more likely to allocate components to low-density regions, including, in the worst case, to outliers. In our results, increasing K beyond the apparent number of modes slightly decrease test NLL, but it could lead to component splitting, where a single mode is approximated by multiple Gaussians. This can increase flexibility, yet it also increases the risk of overfitting when the training data are contaminated.

We also explored an MoE-inspired variant, motivated by the fact that a standard MDN predicts parameters for all K components at every forward pass. On the stylized circle–star dataset, GatedDual achieves comparable modeling quality with reduced inference cost. However, this experiment is deliberately simplified: we choose (K_A, K_B) by hand and the intended expert specialization is only a hypothesis. A more convincing evaluation would require systematic sweeps over (K_A, K_B), repeated runs across seeds, and diagnostics such as responsibilities or component-wise visualizations to verify whether specialization actually emerges.

The main limitation of this study is scale: we rely on small datasets and relatively modest networks, which limits the strength of conclusions about overfitting and generalization. Future work should include larger-scale ablations (more data, larger backbones) and explicit regularization tailored to mixtures, such as entropy regularization on the gating distribution to enhance robustness to outliers. Finally, MDNs are relevant in realistic settings where multimodal predictions and calibrated uncertainty matter, such as conditional image synthesis [9] and motion forecasting [10]. A natural next step is to deploy MDNs on concrete large-scale tasks and evaluate their accuracy and compute trade-offs under engineering constraints.

5 Contributions

The work was distributed as follows:

- **Gloire LINVANI:** Conducted Experiments 1, 5, implemented the baseline MLP and standard MDN architecture.
- **Ying JIN:** Designed and conducted Experiments 2, 3 and 4, designed and implemented the GatedDual architecture, set up the theoretical part and completed result analysis and discussion.

We both contributed equally to writing the report.

References

[1] C. M. Bishop. Mixture Density Networks. Technical Report NCRG/94/004, Neural Computing Research Group, Aston University, 1994.

[2] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[3] M. I. Jordan and R. A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994.

[4] K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, Cambridge, MA, 2022.

[5] G. J. McLachlan and K. E. Basford. *Mixture Models: Inference and Applications to Clustering*. Marcel Dekker, New York, 1988.

[6] A. Q. Jiang et al. Mixtral of experts. arXiv preprint arXiv:2401.04088, 2024.

[7] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[8] N. Du et al. GLaM: Efficient scaling of language models with mixture-of-experts. In *ICML*, 2022.

[9] S. Guadarrama et al. PixColor: Pixel recursive colorization. In *BMVC*, 2017.

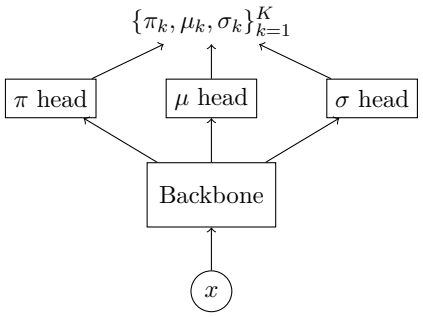
[10] T. Salzmann et al. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *ECCV*, 2020.

[11] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng. Generating Diverse and Natural 3D Human Motions from Text. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5142–5151, 2022.

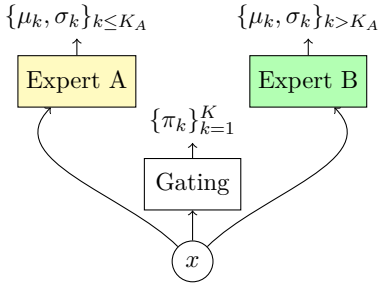
A Models

A.1 MLP Regressor

In the first experiment, we use a standard feedforward network with two hidden layers as a baseline model. The model is trained by minimizing MSE loss, using Adam with learning rate 10^{-3} and batch size 128 for 2000 epochs. This provides a deterministic point prediction $\hat{y} = f_\theta(x)$ approximating $\mathbb{E}[y|x]$.



(a) MDN: shared backbone with separate heads.



(b) GatedDual: gating with experts for different K.

Figure 5: Comparison of MDN architectures. (a) Base MDN architetcure processes all mixture components through a shared backbone. (b) GatedDual employs a gating network and two specialized expert networks for improved efficiency.

A.2 MDN architecture

Our base MDN architecture employs a shared backbone network followed by separate output heads for mixture parameters. The Backbone is designed with two hidden layers with 64 units each(initialized and adapt by experiments) and ReLU activations. Three linear transformations(trainable parameters) are used:

- **Gating head:** Linear layer producing K logits $\{a_k\}$, transformed via softmax to obtain mixing weights $\{\pi_k\}$.
- **Mean head:** Linear layer producing $K \times c$ values, reshaped to $\{\mu_k\} \in \mathbb{R}^{K \times c}$.
- **Scale head:** Linear layer producing $K \times c$ values, passed through softplus function to obtain positive scales $\{\sigma_k\} \in \mathbb{R}^{K \times c}$.

See Figure 5(a). For 1D outputs with K components, this architecture produces $3K$ output values per input. Training proceeds by minimizing the NLL (6) loss.

A.3 New Architecture: GatedDual

Standard MDN architectures compute parameters for all K mixture components at every forward pass during inference. For large K, this becomes computationally expensive. To improve computational efficiency, we propose an architecture, **GatedDual**, that dynamically routes inputs through expert networks guided by the output of the gating network:

- **Gating network:** A network that produces the mixture weights $\{\pi_k\}$. The architecture is three hidden layers with 96 hidden units each
- **Expert A:** A smaller expert network responsible for K_A mixture components. Architecture: input $\to$ 64 hidden units (ReLU) $\to$ $K_A \times 2c$ outputs (means and scales).
- **Expert B:** A larger expert network responsible for K_B mixture components. Architecture: input $\to$ 64 hidden units (ReLU) $\to$ $K_B \times 2c$ outputs.

where $K_A + K_B = K$. The gating network and the experts take the same input x but have independent parameters. At inference time, we evaluate only the expert(s) selected by the gating network and assemble the resulting parameters into a K-component mixture, preserving the standard MDN form. See Figure 5(b).

B Derivation of MDN Gradients

We derive the gradients of the per-sample MDN loss $\ell(\theta; x, y) = -\log p_\theta(y \mid x)$ with respect to the raw network outputs $\{m_k(x;\theta), s_k(x;\theta), a_k(x;\theta)\}$. These gradients are then backpropagated through the network to update the parameters θ.

B.1 Notation

For a single training pair (x, y), we define:

$$\begin{aligned} \phi_k(\theta; x, y) &= \mathcal{N}(y \mid \mu_k(x;\theta), \sigma_k^2(x;\theta) I) \\ Z &\equiv \sum_{j=1}^{K} \pi_j(x;\theta)\ \phi_j \\ \ell(\theta; x, y) &\equiv -\log Z \end{aligned}$$

For brevity, we write $\phi_k(\theta; x, y) = \phi_k$, $\ell(\theta; x, y) = \ell$, $\mu_k(x;\theta) = \mu_k$, $\sigma_k(x;\theta) = \sigma_k$ and $\pi_k(x;\theta) = \pi_k$. The responsibility of component k is:

$$\gamma_k(x, y; \theta) = \frac{\pi_k \phi_k}{Z} \tag{7}$$

B.2 Gradient with respect to m_k (mean)

By the chain rule:

$$\frac{\partial \ell}{\partial m_k} = -\frac{1}{Z}\frac{\partial Z}{\partial m_k} = -\frac{1}{Z}\pi_k \frac{\partial \phi_k}{\partial m_k} \tag{8}$$

For an isotropic Gaussian in dimension c, $\phi_k = (2\pi)^{-c/2}\sigma_k^{-c}\exp\left(-\frac{\|y-\mu_k\|^2}{2\sigma_k^2}\right)$, we have:

$$\frac{\partial \log \phi_k}{\partial \mu_k} = \frac{y - \mu_k}{\sigma_k^2} \quad \Rightarrow \quad \frac{\partial \phi_k}{\partial \mu_k} = \phi_k \frac{y - \mu_k}{\sigma_k^2} \tag{9}$$

Substituting:

$$\frac{\partial \ell}{\partial m_k} = -\frac{\pi_k \phi_k}{Z}\frac{y - \mu_k}{\sigma_k^2} = \gamma_k(x, y; \theta)\frac{\mu_k - y}{\sigma_k^2} \tag{10}$$

Since $\mu_k = m_k$, this is the gradient with respect to m_k.

B.3 Gradient with respect to s_k (log-scale)

We have $\sigma_k = \exp(s_k)$, so:

$$\frac{\partial \ell}{\partial \sigma_k} = -\frac{1}{Z}\frac{\partial Z}{\partial \sigma_k} = -\frac{1}{Z}\pi_k \frac{\partial \phi_k}{\partial \sigma_k} \tag{11}$$

The log-density is:

$$\log \phi_k = -\frac{c}{2}\log(2\pi) - c \log \sigma_k - \frac{\|y - \mu_k\|^2}{2\sigma_k^2} \tag{12}$$

Differentiating:

$$\frac{\partial \log \phi_k}{\partial \sigma_k} = -\frac{c}{\sigma_k} + \frac{\|y - \mu_k\|^2}{\sigma_k^3} \tag{13}$$

Thus:

$$\frac{\partial \phi_k}{\partial \sigma_k} = \phi_k \frac{\partial \log \phi_k}{\partial \sigma_k} = \phi_k \left(-\frac{c}{\sigma_k} + \frac{\|y - \mu_k\|^2}{\sigma_k^3}\right) \tag{14}$$

and:

$$\frac{\partial \ell}{\partial \sigma_k} = \gamma_k(x, y; \theta) \left(\frac{c}{\sigma_k} - \frac{\|y - \mu_k\|^2}{\sigma_k^3} \right) \tag{15}$$

By the chain rule:

$$\frac{\partial \ell}{\partial s_k} = \frac{\partial \ell}{\partial \sigma_k} \frac{\partial \sigma_k}{\partial s_k} = \frac{\partial \ell}{\partial \sigma_k} \sigma_k \tag{16}$$

So:

$$\frac{\partial \ell}{\partial s_k} = \gamma_k(x, y; \theta) \left(c - \frac{\|y - \mu_k\|^2}{\sigma_k^2} \right) \tag{17}$$

B.4 Gradient with respect to a_k (gating logit)

We have $\pi_k = \text{softmax}(a)_k$. First:

$$\frac{\partial \ell}{\partial \pi_k} = -\frac{1}{Z} \frac{\partial Z}{\partial \pi_k} = -\frac{1}{Z} \phi_k = -\frac{\gamma_k(x, y; \theta)}{\pi_k} \tag{18}$$

The Jacobian of softmax is:

$$\frac{\partial \pi_j}{\partial a_k} = \pi_j(\delta_{jk} - \pi_k) \tag{19}$$

where δ_{jk} is the Kronecker delta. By the chain rule:

$$\frac{\partial \ell}{\partial a_k} = \sum_{j=1}^{K} \frac{\partial \ell}{\partial \pi_j} \frac{\partial \pi_j}{\partial a_k} \tag{20}$$

$$= \sum_{j=1}^{K} \left(-\frac{\gamma_j(x, y; \theta)}{\pi_j} \right) \pi_j(\delta_{jk} - \pi_k) \tag{21}$$

$$= -\gamma_k(x, y; \theta) + \pi_k \sum_{j=1}^{K} \gamma_j(x, y; \theta) \tag{22}$$

Since $\sum_j \gamma_j(x, y; \theta) = 1$:

$$\frac{\partial \ell}{\partial a_k} = \pi_k - \gamma_k(x, y; \theta) \tag{23}$$

B.5 Experiment 5: Text-conditioned Human Motion Generation with Diffusion and Mixtures

Motivation. A real-world application of Mixture Density Networks (MDNs) is **text-to-motion** generation. For a given caption c, there are typically many valid motion sequences $x_0 \in \mathbb{R}^{T \times d}$ (different speeds, timings, styles), hence the conditional distribution $p(x_0 \mid c)$ is strongly multimodal. As discussed in [1, 4], minimizing an ℓ_2 regression loss yields a conditional mean estimator which can average incompatible modes. This motivates replacing unimodal conditional regressors by mixture models (MoE/MDN) [1, 2, 3] in generative pipelines.

Baseline: text-conditioned diffusion model. We considered a DDPM-style motion diffusion model, where the forward noising process is

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t) I\Big) \tag{24}$$

and the denoiser ϵ_θ is trained with the objective

$$\mathcal{L}_{\text{MSE}}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\big[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2\big] \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon. \tag{25}$$

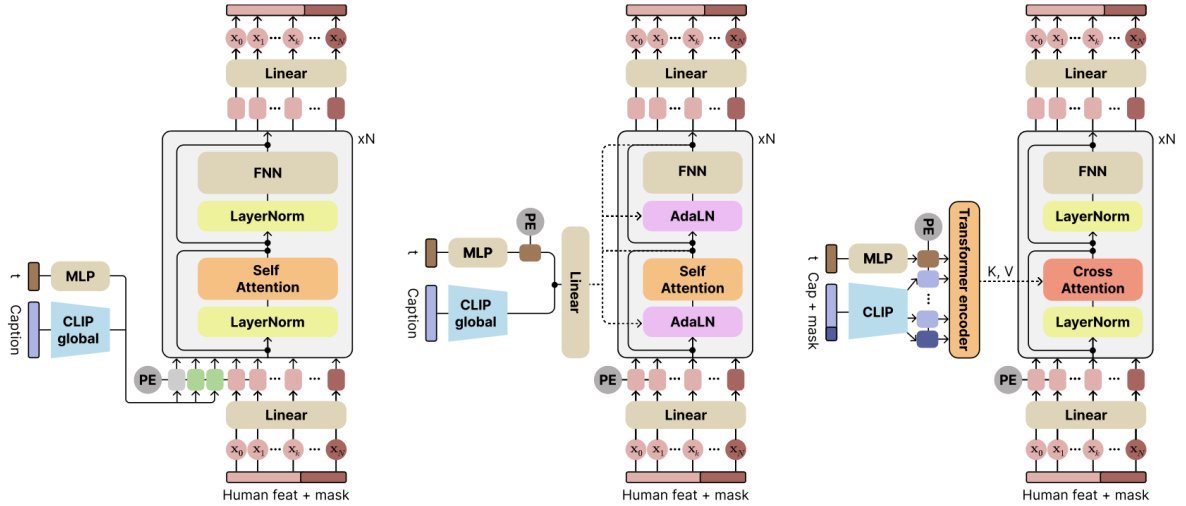


Figure 6: Text-conditioning variants considered for motion diffusion: (left) Incontext, (middle) AdaLN, (right) Cross-attention.

In practice, the architecture follows a DiT-like transformer backbone, with caption conditioning implemented using one of three configurations: (i) **Incontext** (global CLIP embedding injected as additional context), (ii) **AdaLN** (caption-dependent adaptive layer normalization), and (iii) **Cross-attention** (token-level caption encoding used as keys/values). Figure 6 provides schematic diagrams of these conditioning strategies.

Proposed extension: MDN denoising head. Our goal was to replace the unimodal denoiser in (25) by a MDN parameterization that models a multi-modal conditional distribution of the denoising target (noise or clean motion). Concretely, we define a mixture model for the noise:

$$p_\theta(\epsilon \mid x_t, t, c) = \sum_{k=1}^{K} \pi_k(x_t, t, c)\, \mathcal{N}\Big(\epsilon \mid \mu_k(x_t, t, c),\ \sigma_k^2(x_t, t, c) I\Big)\,, \tag{26}$$

where the network outputs mixture weights $\{\pi_k\}_{k=1}^K$, means $\{\mu_k\}$, and positive scales $\{\sigma_k\}$, exactly as in Section 2. The training objective becomes the negative log-likelihood:

$$\mathcal{L}_{\text{MDN}}(\theta) = \mathbb{E}_{t,x_0,\epsilon}\left[-\log \sum_{k=1}^{K} \pi_k(x_t, t, c)\mathcal{N}\Big(\epsilon \mid \mu_k(x_t, t, c), \sigma_k^2(x_t, t, c) I\Big)\right] \tag{27}$$

This turns the denoising step into a conditional mixture model, where the gating network $p_\theta(z = k \mid x_t, t, c) = \pi_k(x_t, t, c)$ routes each noisy motion to experts specialized for different motion modes [2, 3]. At inference time, one may either sample $z \sim \text{Cat}(\pi)$ and then sample ϵ from the corresponding Gaussian, or use mixture-mean estimates corresponding to the most probable mixture component, depending on the desired diversity.

Planned evaluation. We planned to compare the baseline approach against the MDN approach using: (i) qualitative generations for ambiguous captions (multi-modal prompts), and (ii) quantitative text-motion alignment and generated motion distribution metrics. Figure 7 shows qualitative samples produced by the baseline diffusion model.

Challenges and why the experiment was not completed. Despite being well-motivated theoretically, we were not able to produce reliable experimental results before the deadline due to a combination of compute and data availability constraints:

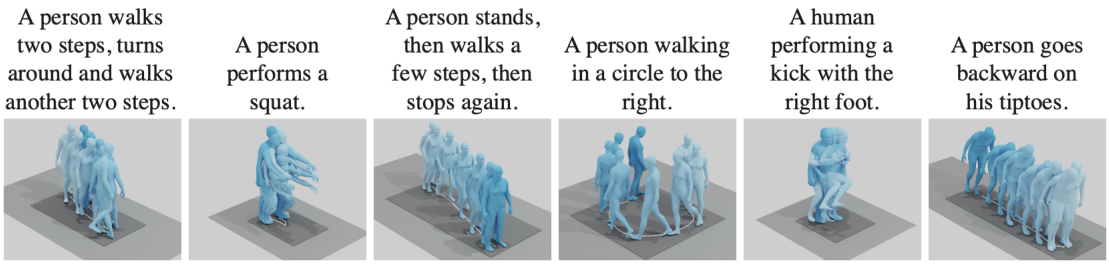


Figure 7: Qualitative motion samples from the text-to-motion pipeline.

- **Training cost of diffusion models.** Unlike Experiments 1–4 (low-dimensional synthetic data), diffusion-based text-to-motion requires large-scale training or at least careful fine-tuning of a high-capacity transformer backbone. With limited GPU access, we could not run enough optimization steps and ablations to obtain meaningful comparisons.
- **Lack of multiple motions per caption in HumanML3D dataset.** HumanML3D [11] provides multiple textual descriptions for each motion clip (3–4 captions per motion), but it does not provide multiple distinct ground-truth motion realizations for the same fixed caption. As a consequence, while MDN modeling is conceptually appropriate for the inherently multi-modal conditional distribution $p(x_0 \mid c)$, it is challenging to quantitatively assess whether mixture components capture different valid modes for a given caption, since the dataset offers limited paired multi-motion supervision and weak multi-reference evaluation for a fixed c.

For these reasons, we include Experiment 5 as a technically grounded proposal and partial implementation plan, but we do not report quantitative results. We view this direction as promising future work: if compute and data permits, the MDN denoiser could better capture the intrinsic multi-modality of $p(x_0 \mid c)$ and potentially improve diversity without sacrificing text-motion coherence.